JOINs in SQL

A **JOIN** in SQL is a clause used to **combine rows from two or more tables** based on a related column between them (typically a primary key–foreign key relationship). It allows relational databases to reconstruct complete datasets from normalized tables.

Types of JOINs Covered

- **INNER JOIN** → returns only matching records in both tables
- **LEFT JOIN** → returns all records from the left table + matches from the right
- **RIGHT JOIN** → returns all records from the right table + matches from the left
- **SELF JOIN** → a table is joined with itself
- **MULTI-JOIN** → joining more than two tables together

Script-by-Script Explanation

1. Database Selection

```
USE techsphere_solutions;
```

What it does:

- Sets the active database context
- Ensures all queries run on the TechSphere schema

2. Base Table Inspection

```
SELECT * FROM employees;
SELECT * FROM employee_positions;
SELECT * FROM job_roles;
SELECT * FROM departments;
```

What it does:

- Displays raw data from all core tables
- Helps understand structure before joining

3. INNER JOIN (Employees + Departments)

```
SELECT *
FROM employees e
INNER JOIN departments d
ON e.department_id = d.department_id;
```

What it does:

- Matches employees to their departments
- Only returns employees who have a valid department

4. INNER JOIN (Employees + Roles via Position Table)

```
SELECT *
FROM employees e
JOIN employee_positions ep
ON e.employee_id = ep.employee_id
JOIN job_roles jr
ON ep.role_id = jr.role_id;
```

What it does:

- Links employees to their job roles
- Uses employee_positions as a bridge table
- Produces full employment-role mapping

5. LEFT JOIN (All Employees)

```
SELECT *
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.department_id;
```

What it does:

- Returns all employees
- Shows department data when available
- Missing departments appear as NULL

6. RIGHT JOIN (All Departments)

```
SELECT *
FROM employees e
RIGHT JOIN departments d
ON e.department_id = d.department_id;
```

What it does:

- Returns all departments
- Includes employees only where a match exists
- Ensures no department is excluded

7. SELF JOIN (Employee Pairing)

```
SELECT *
FROM employees e
RIGHT JOIN departments d
ON e.department_id = d.department_id;
```

What it does:

- Joins the employees table to itself
- Creates artificial relationships (e.g., mentor–mentee pairing)
- Matches consecutive employee IDs

8. MULTI-TABLE JOIN

```
SELECT *
FROM employees e
INNER JOIN departments d
ON e.department_id = d.department_id
INNER JOIN employee_positions ep
ON e.employee_id = ep.employee_id
INNER JOIN job_roles jr
ON ep.role_id = jr.role_id;
```

What it does:

- Combines 4 related tables:
  - Employees
  - Departments
  - Positions
  - job roles
- Produces a complete organizational dataset

9. LEFT JOIN with Multiple Tables

```
SELECT *
FROM employees e
INNER JOIN employee_positions ep
ON e.employee_id = ep.employee_id
LEFT JOIN departments d
ON e.department_id = d.department_id;
```

What it does:

- Ensures all employees appear
- Adds department data when available
- Keeps unmatched employees in results

Key Learning Outcomes

After this script, you understand:

- How relational tables connect using keys
- Differences between INNER, LEFT, RIGHT JOINs
- How SELF JOINs simulate relationships within a table
- How to combine multiple tables for full business datasets
- How JOINs are used in real-world reporting systems

Summary Insight

JOINs are the foundation of relational databases. They allow fragmented data stored across multiple tables to be reconstructed into meaningful business information.